Open in app ↗ Sign up Sign in

Medium Search

Affirm Tech Blog · Follow publication

Machine Learning + Artificial Intelligence + Transformers + Fintech +

Software Engineering +

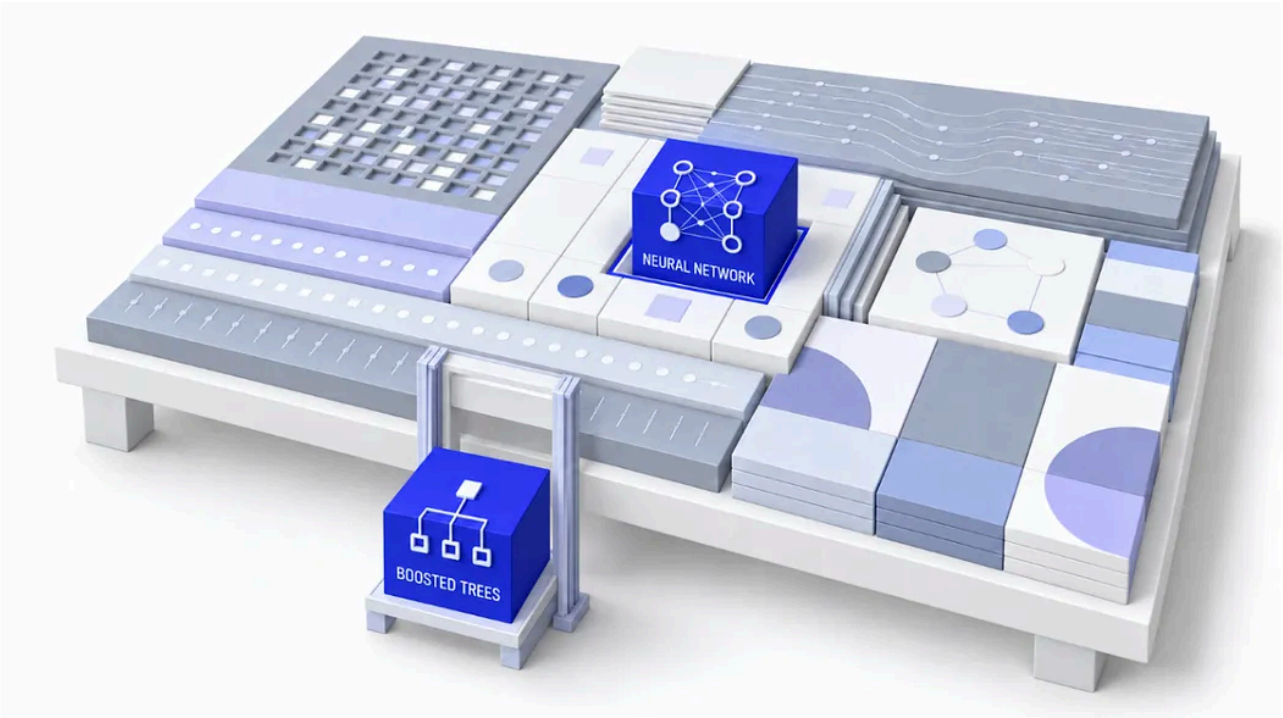
The Model Is Not the Modeling

10 min read · Sep 29, 2026

Affirm Technology Follow

Listen Share

Author: Alexander Statnikov, Machine Learning and Decision Platform, Affirm



A familiar experiment for ML teams in financial services starts with a mature XGBoost classifier, replaces it with a neural network model, and asks whether the metric improves. That is a useful experiment. At Affirm, our experience building

production ML systems has shown the value of looking beyond the classifier. A model swap answers one specific question: how well does another model learn from the representation we already built?

In a mature system, a feature table embodies years of modeling decisions: what constitutes an observation, which events to aggregate, how to represent time, and which outcomes to predict. XGBoost is strong at learning from these tables. A classifier swap keeps those modeling decisions in place. Neural network models can learn representations from detailed records, sequences, and relationships, or share learning across related tasks. Doing this well requires designing the data, targets, and model together.

Why this question matters now

Deep learning's success in language and vision, together with production applications such as our transformer underwriting work, gives ML teams in financial services a reason to revisit established XGBoost systems. AI coding agents can lower the effort required to implement unfamiliar architectures, while a growing set of pretrained tabular foundation models provides candidates that can be evaluated without training from scratch.

That makes a classifier swap an attractive first step. It also makes it tempting to treat that convenient experiment as the whole modeling opportunity. A swap that delivers meaningful, repeatable gains under production constraints may be exactly the right answer. But a disappointing result does not rule out value from different inputs, representations, or training objectives. As trying a new model becomes easier, teams still need to decide which parts of the system to change to improve business decisions.

Start with the behavior and decision

A prediction problem begins with the process being modeled and the decision it will support. What is known at that moment? Over what horizon will the outcome unfold? These questions determine the observation unit, target, and evaluation.

In financial services data, timing is part of that definition. For a fixed-horizon target, an account without sufficient follow-up does not have an observed negative outcome. A join against a current customer record can introduce facts unavailable at the original decision, giving an offline evaluation information the deployed model would not have had.

The lending policy also shapes the available evidence. For credit models trained on historical loan outcomes, repayment labels describe loans booked under the approval policy in place at the time; they do not show how declined applications would have performed if approved. Predicting outcomes under an existing policy and estimating the effect of changing that policy are different questions. Predictive accuracy alone cannot establish the effect of an intervention.

Consider underwriting a new buy now, pay later (BNPL) loan at checkout using the customer's credit report. For this illustration, the target is whether the new loan's first scheduled installment reaches DQ60, meaning at least 60 days delinquent. Figure 1 shows one revolving tradeline for each of two synthetic customers, with all shown history available at checkout. The same prediction target applies to both customers; their future BNPL outcomes are not shown.

Both tradelines have the same utilization values in a different order, share the selected summaries in the figure, and are current in their latest available snapshot.

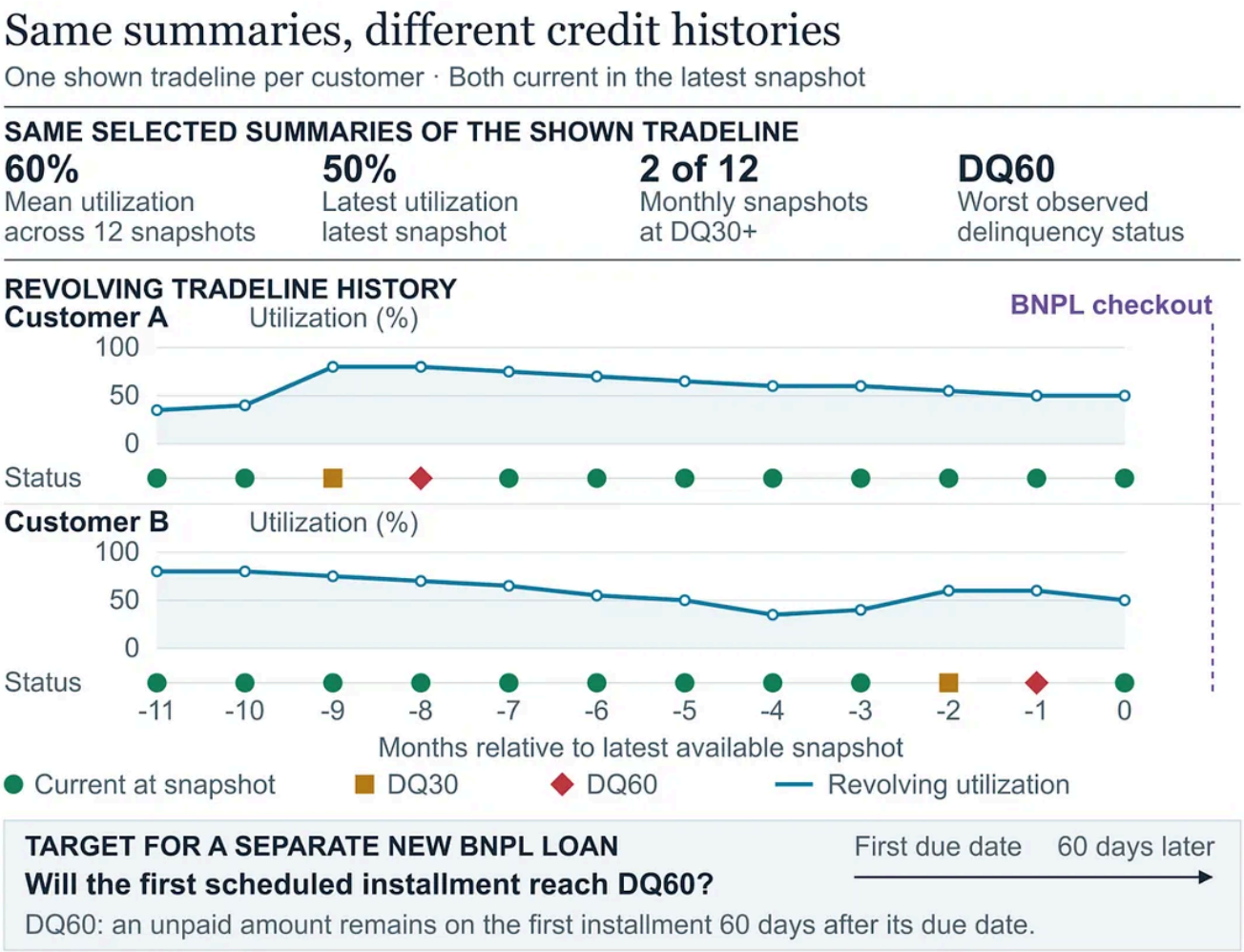


Figure 1. Synthetic revolving-tradeline histories as inputs to a new BNPL loan's first-payment DQ60 prediction. Utilization is balance divided by credit limit; both tradelines have the same fixed limit and 12 monthly observations available at checkout. Current, DQ30, and DQ60 describe the tradeline at each

observation, not individual payment events. The separate target below the histories applies to both customers.

If those summaries were the entire input, changing the classifier could not recover the missing chronology. A credit modeling specialist would look at how delinquency and utilization evolved together. Customer A's episode is followed by eight current monthly observations and declining utilization. Customer B's utilization rises ahead of a recent episode, then falls as the account returns to current. The latest status is the same, but the length of the subsequent current period and the utilization trajectories differ.

These observations suggest useful summary features for a strong XGBoost baseline: delinquency recency, time since returning to current, and utilization trends before and after the episode. The modeling question is whether learning from detailed credit-report histories adds predictive value beyond those summary features. A learned representation may extract useful temporal patterns from the histories that the chosen summaries do not preserve; whether that improves prediction of BNPL repayment must be established in evaluation.

Figure 1 shows one revolving tradeline per customer. A credit report can contain many, with different balances, limits, payment histories, and reporting dates. Teams must decide whether to retain all available tradelines or select a subset, how much history to include, and how to represent the accounts together. These choices determine what information both engineered features and learned representations can use.

Make the representation hypothesis explicit

The example suggests three experiments, summarized in Figure 2. A classifier swap tests another estimator on the existing representation. Improving temporal features while retaining XGBoost tests the value of making that structure explicit. Learning from the histories tests whether a learned representation adds value beyond a strong engineered baseline, potentially in combination with XGBoost. The first experiment cannot stand in for the other two.

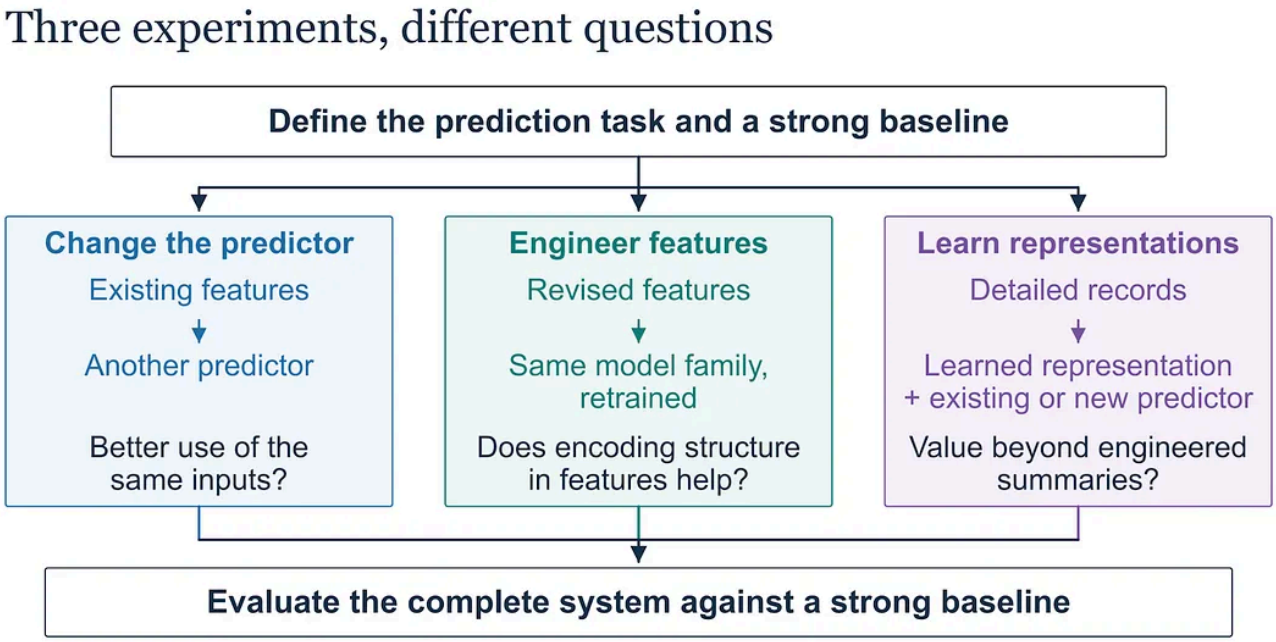


Figure 2. Three complementary experiments that answer different questions. The paths can be combined; they are not required stages.

Learned representations offer a way to discover and combine patterns without specifying every useful summary in advance. Their value is an empirical question even in a mature use case. A useful hypothesis identifies the structure the model could capture, why it might improve prediction, and what result would challenge that expectation. That structure might be payment sequences, relationships across accounts, or behavior shared across related tasks.

Our transformer underwriting work illustrates the third approach. A transformer learns representations from detailed credit-report records and payment histories; XGBoost combines those representations with established features. Our recent blog post explains the modeling choices, production engineering, and results in detail.

Representation choices also affect how a system can respond to change. Repeatedly choosing windows, thresholds, and feature combinations against the same development data can overfit those choices. Separately, fixed summaries may encode assumptions about behavior that become less useful as products or populations change. With detailed histories as inputs, retraining on newer data can update both the learned representation and the predictor, potentially reducing manual feature redesign. Neural network models can also learn unstable relationships. A fair comparison measures both performance under shift before an update and recovery after retraining, giving each approach comparable access to newer data.

Use diagnostics to earn additional complexity

A simple baseline for the final outcome makes it easier to judge whether added complexity helps. For a neural network model, poor training fit and poor generalization call for different investigations. When training fit is poor, checks of the target and loss, input scaling and skew, missing-value handling, and optimization come before adding capacity. Strong training fit with weak validation performance shifts the investigation toward overfitting or differences in time periods or populations.

A problem in a small part of a network can disrupt the entire training run. When training stalls or becomes unstable, activation and gradient magnitudes by block can reveal vanishing signals, abrupt growth, or numerical errors. Controlled changes to normalization, residual scaling, or optimization can then test the suspected cause. These checks help distinguish a failure in the training setup from a failure of the modeling hypothesis.

Get Affirm Technology's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Remember me for faster sign in

A chain of component models can serve useful purposes, yet individually accurate components can still produce a weak final estimate. Evaluating the combined prediction and the cost of maintaining the chain can expose problems that component metrics miss. If the system were being designed today, would the same decomposition still make sense? A direct prediction or a simpler combination may offer a better starting point.

Additional complexity is useful when it lets the model learn something relevant to the final prediction. In repayment modeling, for example, early outcomes can provide additional supervision for a representation used to predict longer-horizon risk. Related tasks may share that representation, while distinct observed behaviors may benefit from specialization. An intermediate objective can improve without

improving the final prediction, so each addition needs to be evaluated by its effect on that final prediction.

Model-only experiments remain valuable. They build familiarity with training and optimization and may improve results on existing features. They become misleading when a limited experiment is treated as a verdict on a much broader modeling opportunity.

Make the comparison answer the question

What information can the model use?

Pretrained tabular foundation models also deserve evaluation on existing feature tables. TabPFN, TabICL, and Google's TabFM apply in-context learning: they use labeled examples supplied at inference time, without requiring parameter updates for each task. Features and labeled context rows define what task-specific information the model can use. Pretraining changes how a model learns a new task; feature selection, context design, and validation remain modeling decisions.

Architecture also determines how features and labeled context can interact. TabPFN v2, for example, alternates attention among feature representations within a row with attention across rows for corresponding feature representations. These operations serve complementary purposes: combining an example's attributes and drawing on the labeled context.

What do the baselines tell us?

Our internal evaluations of pretrained tabular models illustrated why one comparison is rarely enough. Boosted trees and ensembles were strongest overall, while some pretrained approaches showed promise with limited labeled data. When features, targets, and model combinations change together, further experiments are needed to understand which changes helped.

A useful first experiment evaluates the pretrained model without parameter updates against two XGBoost baselines:

- **Matched-sample baseline.** Compare with XGBoost trained on the same labeled rows to assess how effectively each method uses that task-specific data.
- **Mature full-data baseline.** Keep the mature full-data XGBoost baseline as the production reference to assess whether the candidate improves on the existing system.

For the matched-sample comparison, use the same target, curated source features, and held-out cases, allowing appropriate preprocessing and training or inference for each method. Retrain XGBoost whenever context size or selection changes, using the corresponding labeled rows, and repeat the comparison across predefined segments. Pretraining and other differences prevent attributing the results to architecture alone.

What changes when the model uses richer inputs?

A comparison using only the summary features illustrated in Figure 1 cannot test whether learning from detailed histories improves prediction. The same principle applies to other event sequences or relationships: the experiment must give the model access to the structure it is meant to evaluate.

Keep the prediction task, information cutoff, evaluation population, and success criteria aligned. Make differences in available information and development effort explicit. Use targeted ablations where practical to understand what helped, while evaluating whether the complete approach is worth pursuing.

Does the evaluation reflect the intended deployment?

If an account appears in multiple snapshots, adjacent observations may share much of their outcome window. For a retrospective deployment test, training labels must have been observable when the model was fit. Split dates, label maturity, and any grouping by account or customer should reflect what will actually be known and whom the model will score. A test on future observations of existing customers answers a different question from a test on entirely new customers.

Configuration choices belong in training and validation, with a record of the alternatives tried and how they were selected. Repeated optimization against validation data can overfit the model-selection process itself. Keeping the held-out test set outside that loop preserves it for the final comparison, alongside temporal or population checks appropriate to the intended deployment.

Turn evidence into the next modeling decision

The value of a measured gain depends on whether it is repeatable, meaningful for the decision, and large enough to justify the added complexity. Breakdowns by time period and data availability show where gains and failures concentrate. Calibration matters when decisions depend on probabilities or expected values. Examining cases near consequential decision boundaries and testing sensitivity to plausible input changes can reveal weaknesses that aggregate metrics conceal. Embeddings,

attention patterns, and feature attributions can suggest hypotheses to investigate; none alone establishes a causal explanation.

Online data availability, latency, maintainability, and explainability determine how much of a measured gain can be realized in production. A richer research model can help establish potential; simplification, precomputation, or distillation can then be evaluated for their tradeoffs. Any compressed or modified model needs its own validation.

An experiment should improve the system or clarify what to try next. A negative result that exposes a target problem or missing information can be more useful than a gain that disappears outside the development set. XGBoost may remain the best predictor, or it may become one part of a stronger system. The work is to make relevant structure learnable and demonstrate its value.

Build with us at Affirm

At Affirm, our ML team builds the systems behind credit underwriting and fraud prevention decisions for millions of consumers. Bringing research into that setting means solving modeling, product, and engineering problems together.

Our transformer underwriting work shows what that can achieve. We brought the transformer model into real-time checkout and are using it to make approval and decline decisions for US based Affirm users.

Getting there required preserving the meaning of credit data from training to online inference, scaling neural network training, and fitting the entire prediction pipeline into a sub-second checkout budget. We also had to explain the model's decisions and design experiments to measure its effect. Solving those problems is how we turn research into better credit decisions for our customers.

If you want to develop new modeling ideas and bring them into production, explore career opportunities at Affirm.

Machine Learning + Artificial Intelligence + Transformers + Fintech +

Software Engineering +

Follow

Written by Affirm Technology

182 followers · 2 following

Follow

Published in Affirm Tech Blog

905 followers · Last published Oct 21, 2026

www.affirm.com/careers